

Reverse ứng dụng Rust cho người mới bắt đầu

Từ mô hình biên dịch và cấu trúc dữ liệu đến phân tích tĩnh, debug, async, FFI và một bài lab tự xây

Rust trở thành native binary như thế nào



Reverse đi ngược từ machine code, dữ liệu và hành vi. Ownership, tên biến, comment và cấu trúc source thường không còn nguyên vẹn.

Đối tượng đọc

Người biết lập trình cơ bản nhưng chưa từng reverse một binary Rust. Không yêu cầu biết assembly trước; tài liệu giới thiệu đủ kiến thức x86-64 để bắt đầu.

Phiên bản 1.0 | Cập nhật tháng 9 năm 2026

Phạm vi sử dụng. Chỉ phân tích chương trình do bạn sở hữu, phần mềm mã nguồn mở, crackme hoặc mục tiêu được cấp phép. Không dùng tài liệu để phá cơ chế bản quyền, lấy dữ liệu trái phép, phát tán mã độc hoặc can thiệp hệ thống của người khác.

Cách dùng tài liệu này

Tài liệu xây theo một vòng lặp: hiểu mô hình Rust tối thiểu, triage binary, tìm một hành vi cụ thể, đọc code quanh hành vi đó và dùng debugger để xác nhận. Bạn không cần đọc toàn bộ binary. Một câu hỏi hẹp như chương trình kiểm tra input ở đâu hoặc chuỗi lỗi này được tạo từ nhánh nào tốt hơn nhiều so với mục tiêu mơ hồ là hiểu hết ứng dụng.

Reverse Rust vẫn là reverse native code. Điểm khác nằm ở dấu vết compiler và cách các khái niệm ngôn ngữ biến thành machine code: generic được monomorphize, closure thành struct ẩn, async thành state machine, trait object thường đi qua vtable, còn panic và formatting kéo theo nhiều mã thư viện chuẩn.

Kết quả sau khi học

- Giải thích được Cargo và rustc tạo PE, ELF hoặc Mach O như thế nào.
- Triage được kiến trúc, section, import, symbol, string và dấu hiệu một binary có khả năng viết bằng Rust.
- Dùng Ghidra để đi từ entry point, string và xref đến logic của ứng dụng.
- Nhận ra biểu diễn thường gặp của slice, String, Vec, Option, Result, enum, trait object và closure mà không phụ thuộc cứng vào một layout không được bảo đảm.
- Đặt breakpoint, quan sát thanh ghi, stack, heap và kiểm chứng giả thuyết với x64dbg, GDB hoặc LLDB.
- Phân biệt mã nghiệp vụ với tiếng ồn từ std, allocator, formatting, panic và generic instance.
- Hoàn thành bài lab tự biên dịch ở debug và release, sau đó đối chiếu kết quả reverse với source.

Mục lục

- 1 Rust tối thiểu cho người reverse
- 2 Từ source đến native binary
- 3 Điều làm binary Rust khác C và C++
- 4 Chuẩn bị môi trường lab
- 5 Triage và nhận diện Rust
- 6 Assembly x86-64 đủ dùng
- 7 Phân tích tĩnh trong Ghidra
- 8 Dữ liệu Rust trong decompiler
- 9 Generic closure trait và vtable
- 10 Panic formatting và chuỗi
- 11 Async Future và state machine
- 12 Phân tích động bằng debugger

- 13 FFI DLL và ứng dụng giao diện
- 14 Strip LTO obfuscation và giới hạn
- 15 Bài lab từ đầu đến cuối
- 16 Lộ trình sáu tuần và tài liệu tham khảo

1 Rust tối thiểu cho người reverse

1.1 Crate module và Cargo

Crate là đơn vị biên dịch cơ bản. Một package Cargo có thể chứa binary crate, library crate và nhiều target. Module chủ yếu tổ chức tên trong source; sau tối ưu, ranh giới module và hàm có thể bị inline hoặc biến mất. Cargo chọn dependency, profile và target, sau đó gọi rustc và linker.

Khái niệm	Trong source	Dấu vết có thể còn
crate và module	Không gian tên và đơn vị build	Tên symbol chưa strip, đường dẫn panic, chuỗi metadata
trait	Hành vi dùng chung	Gọi trực tiếp sau monomorphization hoặc gián tiếp qua vtable
generic	Một định nghĩa cho nhiều type	Nhiều bản machine code chuyên biệt
closure	Hàm vô danh bất biến	Một struct ẩn và hàm call tương ứng
async fn	Hàm trả Future	State machine với trường trạng thái và dữ liệu sống qua await

1.2 Ownership không phải cơ chế runtime

Ownership, borrowing và lifetime chủ yếu được compiler kiểm tra. Binary thường không mang một bảng ownership để bạn đọc. Dấu vết thực tế là nơi giá trị được move hoặc copy, nơi destructor được gọi, đường cleanup khi return hoặc panic và cách con trỏ được truyền. Vì thế đừng cố tìm lifetime trong assembly; hãy tìm vòng đời cấp phát, sử dụng và giải phóng.

- **Move:** có thể chỉ là copy bit hoặc chuyển quyền quản lý; không nhất thiết có một instruction đặc biệt.
- **Borrow:** thường trở thành pointer hoặc pointer cộng metadata; kiểm tra compile time không còn thành nhân runtime.
- **Drop:** compiler chèn cleanup khi giá trị ra khỏi scope; tối ưu có thể inline hoặc loại bỏ nếu không cần.

- **Unsafe:** không có cờ cho toàn hàm ở binary; nó chỉ cho phép source thực hiện thao tác mà compiler không chứng minh an toàn.

1.3 Những type phải nhận ra

Type	Mô hình tinh thần ban đầu	Cảnh báo
&T và Box<T>	Một con trỏ tới T	Quyền sở hữu khác nhau ở source nhưng bit pattern có thể giống
&str và &[T]	Pointer cộng length	Là fat pointer; raw str hoặc slice không tự có metadata ngoài con trỏ
String và Vec<T>	Ba machine word cho buffer, length, capacity	Thứ tự trường thuộc layout compiler; hãy xác nhận ở binary cụ thể
Option và Result	Enum gồm variant và payload	Compiler có thể dùng niche optimization, không luôn có tag riêng
dyn Trait	Data pointer cộng vtable pointer	Chi tiết vtable không phải ABI ổn định công khai

2 Từ source đến native binary

Rust trở thành native binary như thế nào



Reverse đi ngược từ machine code, dữ liệu và hành vi. Ownership, tên biến, comment và cấu trúc source thường không còn nguyên vẹn.

rustc hạ source qua các biểu diễn trung gian, tối ưu và thường dùng LLVM để tạo object code. Linker ghép code của crate, dependency, runtime cần thiết và thư viện native thành file thực thi hoặc thư viện. Kết quả là machine code cho kiến trúc đích, không phải bytecode Rust có thể đổi ngược hoàn hảo.

2.1 Debug và release tạo hai mục tiêu rất khác

Thuộc tính	Debug thường gặp	Release thường gặp
Tối ưu	Thấp hoặc tắt	Cao hơn, inline và biến đổi mạnh
Debug info	Thường có	Tùy profile và packaging
Kiểm tra overflow	Thường bật	Thường tắt theo profile mặc định
Ranh giới hàm	Gần source hơn	Có thể inline, merge hoặc loại bỏ
Kích thước	Lớn vì metadata	Có thể vẫn lớn do static linking và generic

Các thiết lập nên tự thử trong Cargo toml

```
cargo build
cargo build --release

[profile.release]
debug = 1
strip = "none"
lto = "thin"
panic = "abort"
```

Mỗi lựa chọn thay đổi bằng chứng. strip loại symbol hoặc debug info; LTO cho phép tối ưu xuyên crate; panic abort loại đường unwind; debug info giúp Ghidra và debugger khôi phục tên, type và source line tốt hơn.

2.2 Dạng binary và crate type

Crate type hoặc sản phẩm	Đầu ra	Điểm bắt đầu
bin	PE EXE, ELF hoặc Mach O	Entry hệ điều hành rồi runtime Rust và hàm main
cdylib	DLL, SO hoặc dylib cho ngôn ngữ khác	Export có ABI C hoặc hàm được gọi qua loader
staticlib	Thư viện tĩnh	Object được ghép vào binary khác
rlib	Định dạng nội bộ của Rust	Không phải ABI phân phối ổn định cho ngôn ngữ khác

3 Điều làm binary Rust khác C và C++

- **Nhiều mã thư viện:** formatting, iterator, panic, allocator và dependency có thể được link vào cùng binary.
- **Monomorphization:** một generic function có thể xuất hiện nhiều lần cho các type khác nhau.
- **Tên dài:** symbol mangled có thể chứa module, type, trait và hash; demangle giúp định hướng khi symbol còn.
- **Enum tối ưu:** Option con trỏ thường dùng null làm None, nên không nhất thiết thấy tag 0 hoặc 1 riêng.
- **Iterator và closure:** chuỗi combinator có thể được inline thành loop khó liên hệ trực tiếp với source.
- **Panic và format:** để lại chuỗi, file path hoặc call tới core panicking, nhưng cũng tạo nhiều hàm không thuộc logic ứng dụng.

Rust không tự động làm binary không thể reverse. Release tối ưu và strip mới là nguyên nhân chính làm cấu trúc khó đọc. Ngược lại, chuỗi panic, đường dẫn source và symbol chưa strip đôi khi khiến target Rust dễ định hướng hơn binary C đã strip sạch.

4 Chuẩn bị môi trường lab

Công cụ	Mục đích	Gợi ý cho người mới
Rustup Cargo rustc	Tạo target để đối chiếu	Biên dịch cùng source ở nhiều profile
Ghidra	Disassembly, decompiler, xref và rename	Miễn phí, đủ cho toàn bộ tài liệu
Detect It Easy	Nhận diện format, packer và compiler	Chỉ dùng như gợi ý, không coi detector là bằng chứng cuối
PE bear hoặc readelf	Header, section, import và symbol	Chọn theo Windows hoặc Linux
x64dbg	Debug user mode trên Windows	Dễ quan sát GUI và API breakpoint
GDB hoặc LLDB	Debug Linux hoặc macOS	Học một bộ lệnh nhỏ trước
rustfilt hoặc demangler	Đổi symbol Rust sang tên dễ đọc	Nhiều công cụ phân tích đã hỗ trợ tự động

Kiểm tra môi trường và triage ban đầu

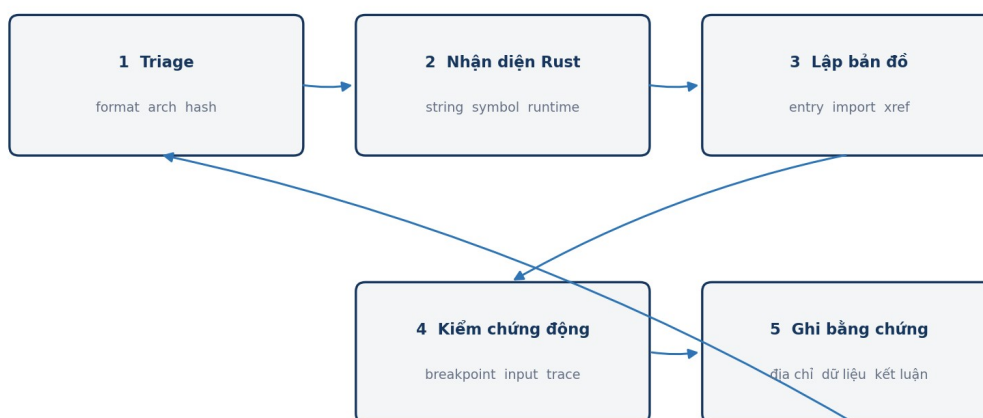
```
rustc --version --verbose
cargo --version
ghidraRun

# Linux
file ./sample
readelf -h -S -s ./sample
objdump -d ./sample
strings -a ./sample | less
```

Chạy target lạ trong máy ảo hoặc sandbox tách biệt, tắt dữ liệu cá nhân và ghi hash trước khi chạy. Static analysis không làm file an toàn; chỉ cần double click nhầm hoặc cho tool thực thi script nhúng cũng có thể gây rủi ro.

5 Triage và nhận diện Rust

Quy trình reverse dựa trên giả thuyết và kiểm chứng



Decompiler giúp đọc nhanh nhưng không phải source. Khi có mâu thuẫn, ưu tiên machine code, bộ nhớ và hành vi runtime.

5.1 Checklist mười phút

- Tính SHA 256; ghi tên file, kích thước, nguồn và thời điểm nhận mẫu.
- Xác định PE, ELF hay Mach O; kiến trúc x86, x86-64, ARM64; endianness và subsystem.
- Liệt kê section, import, export, symbol, debug directory, chữ ký và dấu hiệu packer.
- Trích strings ASCII và UTF 16; tìm thông báo lỗi, URL, đường dẫn, tên file, format string và panic text.
- Chạy detector compiler nhưng coi kết quả là giả thuyết; đối chiếu nhiều dấu hiệu độc lập.
- Chọn một hành vi cụ thể để reverse và ghi cách kích hoạt hành vi đó.

5.2 Dấu hiệu Rust và bẫy nhận diện

Dấu hiệu	Ý nghĩa	Vì sao chưa đủ một mình
Symbol có tên module Rust hoặc hash	Symbol chưa strip và mangling còn	Binary có thể chứa một thư viện Rust nhưng app chính không phải Rust
core panicking hoặc rust begin unwind	Runtime panic Rust hiện diện	Tên thay đổi theo phiên bản và chế độ panic
alloc hoặc dealloc pattern	Dùng allocator Rust	Allocator tương tự cũng có trong ngôn ngữ khác
Đường dẫn crate registry hoặc .rs	Debug hoặc panic metadata còn	Có thể chỉ đến từ dependency
Ít import nhưng binary lớn	Static linking khả dĩ	C và Go cũng có thể có đặc điểm này

Kết luận nên dựa trên cụm dấu hiệu: symbol hoặc chuỗi Rust, cấu trúc runtime, panic path và cách entry đi qua hàm lang start. Detector chỉ là điểm xuất phát.

6 Assembly x86-64 đủ dùng

Bạn không cần thuộc toàn bộ instruction set. Trước hết hãy đọc được dữ liệu đi vào hàm, so sánh nào quyết định branch, con trỏ trỏ tới đâu và giá trị trả về nằm ở đâu. Calling convention phụ thuộc nền tảng và ABI; phải xác nhận target trước khi gán ý nghĩa cho thanh ghi.

Khái niệm	Windows x64	System V AMD64
Đối số integer đầu	RCX RDX R8 R9	RDI RSI RDX RCX R8 R9
Giá trị trả về	RAX	RAX
Stack	Có shadow space; alignment theo ABI	Red zone trong user space; alignment theo ABI
Thanh ghi volatile	Theo quy ước Microsoft	Theo quy ước System V

Những pattern cơ bản cần nhận ra

```
test eax, eax ; đặt cờ theo eax
je fail ; nhảy nếu eax bằng 0
cmp rdx, 0x0c ; so sánh length với 12
jne fail
mov rax, [rcx+8] ; đọc một field hoặc phần tử qua con trỏ
call qword ptr [r10+18h] ; gọi gián tiếp, có thể qua vtable
```

Decompiler tạo if, loop và biến giả từ control flow. Hãy đổi tên biến theo vai trò đã chứng minh như `input_ptr`, `input_len` hoặc `status` thay vì đoán tên `source`. Khi thấy cặp pointer và length đi cùng nhau, hãy thử giả thuyết slice hoặc str rồi kiểm tra nơi chúng được sử dụng.

7 Phân tích tĩnh trong Ghidra

7.1 Import và auto analysis

- Tạo project riêng và import bản sao của mẫu; chọn đúng language và compiler specification.
- Cho auto analysis chạy, nhưng giữ log về lỗi disassembly hoặc vùng chưa được nhận diện.
- Mở Symbol Tree, Defined Strings, Function Graph, Decompiler và References.
- Demangle symbol nếu có; đổi tên hàm và biến theo bằng chứng, thêm comment ghi nguồn của kết luận.
- Tạo type thủ công cho struct đơn giản chỉ sau khi quan sát đủ offset và cách sử dụng.

7.2 Ba con đường vào logic

Con đường	Khi dùng	Cách đi
Entry point	Binary nhỏ hoặc symbol còn	Entry OS đến runtime, lang start và hàm main của crate
String xref	Có thông báo UI, lỗi, file path hoặc URL	String đến xref, caller và branch tạo thông báo
API xref	Hành vi chạm file, mạng, registry hoặc crypto	Import hoặc syscall đến caller rồi lần ngược dữ liệu

Trong binary Rust lớn, entry point thường đưa bạn qua nhiều lớp runtime. String và API xref thường nhanh hơn. Khi tìm được một hàm ứng viên, xem cả caller lẫn callee: caller cho biết input đến từ đâu, callee cho biết dữ liệu được biến đổi thế nào.

7.3 Tách std khỏi logic ứng dụng

- Mã formatting thường có nhiều descriptor, function pointer và xử lý buffer; đừng đọc sâu nếu câu hỏi chỉ cần giá trị được format.
- Iterator adapter sau inline có thể thành loop lớn; tập trung vào phép biến đổi dữ liệu và điều kiện kết thúc.
- Drop glue và unwind cleanup xuất hiện quanh nhiều exit path; đặt tên cleanup để giảm nhiễu.
- Generic instance có logic gần giống nhau; so sánh constant, size và caller để nhận ra vai trò từng bản.

8 Dữ liệu Rust trong decompiler

8.1 Chuỗi slice String và Vec

Một &str về mặt sử dụng thường giống pointer cộng byte length; dữ liệu phải là UTF 8 hợp lệ theo invariant Rust. &[T] cũng mang pointer và số phần tử. String và Vec quản lý buffer heap cùng length và capacity, nhưng không nên áp một thứ tự field cứng cho mọi compiler. Hãy quan sát argument được truyền cho allocator, phép so sánh length với capacity và địa chỉ được giải phóng.

Cách biểu diễn giả thuyết slice mà không giả vờ là source gốc

```
// Pseudocode đã đặt lại tên theo bằng chứng
bool verify(uint8_t *input_ptr, uint64_t input_len) {
    if (input_len != 12) return false;
    return transform(input_ptr, input_len) == EXPECTED;
}
```

8.2 Option Result và niche optimization

Enum Rust có variant và có thể có payload. Tuy nhiên compiler được phép chọn layout tối ưu cho repr Rust. Option<&T> thường dùng null làm None; Option<NonZeroU32> có thể dùng 0 làm None; Result có thể dùng tag riêng hoặc tận dụng giá trị không hợp lệ của payload. Vì vậy một nhánh kiểm tra null có thể chính là match Option, nhưng cần xem caller và downstream để xác nhận.

Quy tắc an toàn. Chỉ khẳng định layout ổn định khi type có repr phù hợp hoặc tài liệu chính thức bảo đảm. Với binary cụ thể, bạn có thể mô tả layout quan sát được tại phiên bản và target đó, không suy rộng thành ABI Rust chung.

8.3 Drop và đường cleanup

Khi một String, Vec, Box hoặc object có Drop ra khỏi scope, compiler chèn cleanup. Trong chế độ unwind, landing pad có thể gọi nhiều destructor trước khi tiếp tục unwind; trong panic abort, nhánh này giảm đáng kể. Một hàm decompile có nhiều goto không nhất thiết có source lộn xộn; chúng có thể là cleanup path do compiler tạo.

9 Generic closure trait và vtable

9.1 Monomorphization

Generic không chạy như một hộp dynamic mặc định. Compiler tạo bản chuyên biệt cho các type thực sự dùng, sau đó optimizer có thể inline. Nếu thấy hai hàm gần giống nhưng khác size, constant hoặc routine drop, hãy thử giả thuyết chúng là hai instance của cùng generic. Symbol demangled, nếu còn, thường giúp xác nhận type parameter.

9.2 Closure

Closure bắt biến được biểu diễn như một object ẩn chứa các capture. Hàm call nhận pointer tới object đó cùng argument. Closure không capture có thể co thành zero sized type hoặc function pointer tùy ngữ cảnh. Layout closure không có bảo đảm ổn định, vì vậy hãy dựng type từ cách các offset được đọc trong chính binary.

9.3 Static dispatch và dynamic dispatch

Kiểu gọi	Dấu hiệu	Cách reverse
Static dispatch	Call trực tiếp hoặc đã inline	Đọc instance cụ thể, đối chiếu symbol và caller
dyn Trait	Call gián tiếp qua bảng function pointer	Tìm data pointer, vtable pointer và các slot được gọi
Function pointer	Call qua thanh ghi hoặc memory	Lần nơi pointer được gán, không mặc định là trait object
C callback	ABI extern C và context pointer	Kiểm tra export, import và quy ước gọi

Một con trỏ kép không tự động là trait object. Slice và str cũng là fat pointer, còn nhiều API C truyền context cộng function pointer. Ý nghĩa đến từ cách word thứ hai được dùng: length tham gia bounds check khác với vtable được dereference thành địa chỉ code.

10 Panic formatting và chuỗi

Panic là mỏ neo tốt cho người mới vì thông điệp, file path và line có thể còn. Tìm các chuỗi như index out of bounds, unwrap thất bại, assertion hoặc thông điệp riêng của ứng dụng; theo xref đến hàm tạo panic rồi lùi lại nhánh lỗi. Tuy nhiên, string xref có thể đi qua bảng descriptor nên Ghidra không luôn tạo reference trực tiếp.

Dấu vết	Có thể suy ra	Không nên suy ra
Đường dẫn src module rs	Vị trí compile time hoặc dependency	File đó còn tồn tại trên máy chạy
unwrap hoặc expect message	Một nhánh lỗi quanh Option hoặc Result	Toàn bộ logic success path
format placeholder	Giá trị được chuẩn bị để in	Tên biến source ban đầu

Dấu vết	Có thể suy ra	Không nên suy ra
panic handler	Điểm tập trung để breakpoint	Mọi lỗi đều đi qua đó khi panic abort hoặc handler tùy biến

Đặt breakpoint tại hàm panic hoặc API in lỗi trong lab giúp bắt thời điểm invariant bị phá. Sau khi dừng, xem stack và caller để quay về logic ứng dụng, không bước từng instruction qua toàn bộ formatting runtime.

11 Async Future và state machine

async fn không chạy toàn bộ khi được gọi; nó tạo một Future. Khi executor poll Future, code chạy đến khi hoàn thành hoặc trả Pending. Compiler giữ những local sống qua await trong một state machine, cùng discriminant trạng thái. Decompiler có thể hiện một switch lớn, nhiều nhánh cleanup và field offset khó hiểu.

- Tìm nơi Future được tạo, nơi nó được poll và executor hoặc runtime nào lái nó.
- Xác định discriminant trạng thái bằng switch hoặc chuỗi so sánh quanh đầu poll function.
- Đánh dấu field nào sống qua await bằng cách xem offset được đọc ở nhiều state.
- Theo dõi I/O boundary như connect, read, write hoặc API runtime thay vì cố đọc toàn state machine từ đầu.
- Nhớ rằng pinning là invariant source và thư viện; không có instruction pin riêng trong CPU.

Với Tokio hoặc runtime khác, phần scheduler có thể rất lớn. Câu hỏi ứng dụng thường nằm ở future cụ thể hoặc callback xử lý kết quả, không nằm trong work stealing queue của executor.

12 Phân tích động bằng debugger

12.1 Chiến lược breakpoint

Breakpoint	Giá trị	Rủi ro nhiều
Hàm main hoặc logic đã tìm	Quan sát input và control flow trực tiếp	Địa chỉ đổi do ASLR; symbol có thể bị strip
API file hoặc network	Bắt hành vi bên ngoài	Nhiều dependency cùng gọi API
Allocator	Theo dõi buffer và object sống	Tần suất rất cao
Panic handler	Dừng đúng nhánh lỗi	Có thể dừng trong lỗi thư viện không liên quan
Chuỗi compare hoặc hash	Kiểm chứng verification path	Optimizer có thể inline hoặc vectorize

12.2 Bộ lệnh GDB LLDB tối thiểu

Các lệnh quan sát cơ bản

```
# GDB
starti
break *0xADDRESS
run
info registers
x/32gx $rsp
x/32bx $rdi
disassemble /m FUNCTION

# LLDB
process launch --stop-at-entry
breakpoint set --address 0xADDRESS
register read
memory read --size 1 --count 32 ADDRESS
thread backtrace
```

ASLR làm địa chỉ runtime khác địa chỉ trong file. Hãy dùng module base cộng relative virtual address hoặc để debugger resolve symbol. Luôn ghi lại input, breakpoint, thanh ghi, bytes bộ nhớ và kết quả; nếu không, bạn dễ biến một quan sát ngẫu nhiên thành kết luận.

12.3 Khi decompiler và runtime mâu thuẫn

- Kiểm tra bạn có đang debug đúng build và đúng architecture không.
- Xem instruction thực tại địa chỉ dừng, không chỉ pseudocode cũ.
- Kiểm tra input thực tế, encoding, length và null termination.
- Xác định optimizer đã constant fold, inline hoặc vectorize hay chưa.
- Ưu tiên kết quả lặp lại được với cùng input; ghi rõ phần còn chưa chắc chắn.

13 FFI DLL và ứng dụng giao diện

13.1 Extern C và ranh giới ABI

Rust ABI mặc định không được thiết kế như ABI ổn định giữa các phiên bản compiler. Khi giao tiếp với C, code thường dùng extern C, no mangle hoặc export name và struct repr C. Đây là điểm reverse thuận lợi vì export và calling convention rõ hơn, nhưng con trỏ vẫn có thể yêu cầu invariant về lifetime, ownership và allocator.

Ví dụ minh họa một ranh giới FFI

```
#[repr(C)]
pub struct Buffer {
    pub ptr: *mut u8,
    pub len: usize,
}

#[unsafe(no_mangle)]
pub extern "C" fn verify(data: *const u8, len: usize) -> bool {
    // boundary dễ thấy trong export table
    true
}
```

Khi thấy một DLL Rust, bắt đầu từ export và các caller. Kiểm tra hàm nào cấp phát và hàm nào giải phóng; giải phóng buffer bằng allocator khác có thể làm crash. Không gán repr C cho type nội bộ chỉ vì nó đi qua một wrapper C.

13.2 GUI và framework

Loại ứng dụng	Logic có thể nằm ở	Mỏ neo phân tích
Win32 hoặc windows rs	Rust gọi API Windows	Window procedure, message ID, import UI và string
egui iced	Event loop và widget state Rust	Label, callback, update function và renderer boundary
Tauri	Rust backend cộng web frontend	Command export, IPC message, JS bundle và URL
Service hoặc CLI	main, parser, worker thread	Argument, config, log, file và socket

14 Strip LTO obfuscation và giới hạn

Cơ chế	Tác động	Cách thích nghi
Strip	Mất symbol và debug info	Dùng string, import, signature, control flow và runtime
LTO	Inline xuyên crate và thay đổi ranh giới hàm	Bám data flow và hành vi thay vì tên hàm
Panic abort	Ít unwind và cleanup path hơn	Tìm abort path hoặc thông điệp trước abort
Mã hóa chuỗi	String scan ít kết quả	Breakpoint sau routine giải mã hoặc tại API nhận plaintext

Cơ chế	Tác động	Cách thích nghi
Packer	Code thật được giải nén khi chạy	Nhận diện loader, dump hợp pháp trong lab và rebuild memory map
Anti debug	Timing, debugger check hoặc exception	Dùng snapshot, log và so sánh nhiều môi trường được phép

Giới hạn quan trọng. Reverse thường không khôi phục được tên biến, comment, cấu trúc module, generic source hay lifetime. Bạn có thể tái dựng hành vi và cấu trúc dữ liệu đủ dùng, nhưng nên phân biệt rõ quan sát, suy luận và phần chưa biết.

Patching chỉ nên thực hành trên binary tự viết hoặc crackme được phép. Thay một branch có thể phá checksum, chữ ký, unwind metadata hoặc invariant của chương trình. Mục tiêu học tập tốt hơn là hiểu điều kiện, chứng minh input nào đi qua success path và tái hiện thuật toán bằng một chương trình độc lập.

15 Bài lab từ đầu đến cuối

15.1 Tạo target hợp pháp

Source target mẫu

```
use std::io::{self, Write};

fn score(s: &str) -> u32 {
    let mut x = 0x1357_9bdfu32;
    for (i, b) in s.bytes().enumerate() {
        x = x.rotate_left(5) ^ (b as u32).wrapping_add((i as u32) * 17);
    }
    x
}

fn verify(s: &str) -> bool {
    s.len() == 12 && score(s) == 0x4f0c_0ee5
}

fn main() {
    print!("Key: ");
    io::stdout().flush().unwrap();
    let mut input = String::new();
    io::stdin().read_line(&mut input).unwrap();
    let input = input.trim_end();
    println!("{}", if verify(input) { "OK" } else { "NO" });
}
```

Để có key đúng, bạn có thể đổi constant thành score của một chuỗi 12 byte do chính bạn chọn, in score một lần trong build debug rồi xóa lệnh in trước khi reverse. Mục tiêu bài lab không phải

đoán key bằng brute force mà là lần được đường data từ read line qua trim, length check, loop rotate XOR và branch in OK.

Biên dịch ba biến thể để so sánh

```
cargo new rust_rev_lab
# thay src/main.rs bằng source ở trên
cargo build
cargo build --release

# tạo thêm biến thể
RUSTFLAGS="-C strip=symbols" cargo build --release
```

15.2 Nhiệm vụ phân tích

- Ghi hash, format, architecture, kích thước và số symbol của từng build.
- Tìm chuỗi Key, OK và NO; theo xref đến nhánh in kết quả.
- Nhận diện length check bằng constant 12 và xác định pointer cộng length của &str.
- Tìm loop đọc từng byte, phép rotate left 5, cộng index nhân 17 và XOR.
- Đổi tên hàm và biến trong Ghidra; ghi comment cho từng bằng chứng.
- Đặt breakpoint trước compare cuối; chạy hai input và ghi RAX hoặc flags cùng accumulator.
- So sánh debug, release và stripped release: hàm nào biến mất, string nào còn, loop nào bị tối ưu.
- Cuối cùng mới mở source và đánh dấu kết luận nào đúng, sai hoặc thiếu.

15.3 Mẫu ghi chép

Mục	Ví dụ cần ghi
Giả thuyết	Hàm tại RVA X là verify vì được gọi trước chuỗi OK hoặc NO
Bằng chứng tĩnh	Có compare length 12 và gọi loop dùng rotate XOR
Bằng chứng động	Breakpoint nhận pointer input và length đúng với dữ liệu nhập
Độ tin cậy	Cao, trung bình hoặc thấp cùng lý do
Việc tiếp theo	Xác nhận constant cuối hoặc dựng lại thuật toán

16 Lộ trình sáu tuần và tài liệu tham khảo

Tuần	Trọng tâm	Sản phẩm đầu ra
1	PE hoặc ELF, x86-64, calling convention	Đọc được branch, call, stack và argument
2	Cargo, rustc, profile và symbol	So sánh debug với release của app tự viết
3	Ghidra, string, xref và rename	Bản đồ entry, I O và logic chính
4	String Vec Option Result và Drop	Tái dựng hai struct hoặc enum có kiểm chứng
5	Debugger, breakpoint và memory	Nhật ký ba thí nghiệm có thể lặp lại
6	Trait object, async, FFI và target lớn hơn	Báo cáo ngắn phân biệt quan sát và suy luận

Cheat sheet khi bị kẹt

- **Không tìm thấy main:** đi từ entry, lang start, symbol nếu có hoặc từ chuỗi do main in ra.
- **Decompiler quá rối:** thu hẹp về một branch, xem graph và assembly, đổi type pointer và length.
- **Không có string xref:** tìm địa chỉ string trong data descriptor hoặc breakpoint tại API in.
- **Quá nhiều hàm giống nhau:** nghi generic instance; so caller, constant, size và drop routine.
- **Call gián tiếp:** lần nơi function pointer hoặc vtable được nạp, không bước mù vào mọi target.
- **Debug lệch địa chỉ:** tính module base cộng RVA và kiểm tra ASLR.
- **Async khó đọc:** tìm I O boundary và state discriminant thay vì đọc scheduler.

Nguồn chính thức nên đọc tiếp

Rust Reference về type layout <https://doc.rust-lang.org/reference/type-layout.html>

Cargo Book về build profiles <https://doc.rust-lang.org/cargo/reference/profiles.html>

rustc book về symbol mangling <https://doc.rust-lang.org/rustc/symbol-mangling/index.html>

Rust Reference về linkage và crate type <https://doc.rust-lang.org/reference/linkage.html>

The Rust Programming Language về async và Future <https://doc.rust-lang.org/book/ch17-05-traits-for-async.html>

Rust standard library về panic <https://doc.rust-lang.org/std/panic/index.html>

Ghidra chính thức <https://github.com/NationalSecurityAgency/ghidra>

Tài liệu x64dbg <https://help.x64dbg.com/en/latest/>

Hướng dẫn LLDB <https://lldb.llvm.org/use/tutorial.html>